

Bucles

time: $O(n)$, space: $O(1)$ | Repetir instrucciones un número de veces o hasta una condición.

```
// AUTO-GENERATED from loops.v1.clean.cpp -- do not edit.
// To override, replace this file with a hand-written version (remove this marker).
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    long long sum = 0;
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        sum += x;
    }
    cout << sum << "\n";
    return 0;
}
```

Búsqueda binaria en un array

time: $O(\log n)$, space: $O(1)$ | Encontrar un valor en un array ordenado.

```
// AUTO-GENERATED from binary_search.v1.clean.cpp -- do not edit.
// To override, replace this file with a hand-written version (remove this marker).
#include <vector>
using namespace std;
int binary_search_idx(const vector<int>& a, int target) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Árbol de Fenwick

build: $O(n)$, update: $O(\log n)$, query: $O(\log n)$, space: $O(n)$ | Sumas de prefijos con actualizaciones puntuales.

```
// AUTO-GENERATED from fenwick.v1.clean.cpp -- do not edit.
```

```
// To override, replace this file with a hand-written version (remove this marker).
```

```
#include <vector>
using namespace std;
struct FenwickTree {
    int n;
    vector<int> tree;
    FenwickTree(int n) : n(n), tree(n) {}
    void update(int index, int delta) {
        for (index++; index <= n; index += index & -index)
            tree[index - 1] += delta;
    }
    int query(int index) {
        int acc = 0;
        for (; index > 0; index -= index & -index)
            acc += tree[index - 1];
        return acc;
    }
    int query_range(int l, int r) { return query(r) - query(l); }
};
```

DP con máscaras de bits — TSP

time: $O(2^n * n^2)$, space: $O(2^n * n)$ | Ruta mínima que visita todos los nodos una vez (n pequeño, $n \leq 20$).

```
// AUTO-GENERATED from tsp.v1.clean.cpp -- do not edit.
// To override, replace this file with a hand-written version (remove this marker).
#include <vector>
#include <algorithm>
using namespace std;
const long long INF = 1e18;
long long tsp(const vector<vector<long long>&& dist) {
    int n = dist.size();
    vector<vector<long long>> dp(1 << n, vector<long long>(n, INF));
    dp[1][0] = 0;
    for (int mask = 1; mask < (1 << n); mask++) {
        for (int u = 0; u < n; u++) {
            if (dp[mask][u] == INF || !(mask & (1 << u))) continue;
            for (int v = 0; v < n; v++) {
                if (mask & (1 << v)) continue;
                int nmask = mask | (1 << v);
                dp[nmask][v] = min(dp[nmask][v], dp[mask][u] + dist[u][v]);
            }
        }
    }
}
```

```
}
long long ans = INF;
int full = (1 << n) - 1;
for (int u = 0; u < n; u++)
    ans = min(ans, dp[full][u] + dist[u][0]);
return ans;
}
```

Envolvente convexa

time: $O(n \log n)$, space: $O(n)$ | El menor polígono convexo que contiene un conjunto de puntos.

```
// AUTO-GENERATED from convex_hull.v1.clean.cpp -- do not edit.
// To override, replace this file with a hand-written version (remove this marker).
#include <vector>
#include <algorithm>
using namespace std;
struct P {
    long long x, y;
    bool operator<(const P& o) const {
        return x < o.x || (x == o.x && y < o.y);
    }
};
long long cross(const P& O, const P& A, const P& B) {
    return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
}
vector<P> convex_hull(vector<P> pts) {
    int n = pts.size();
    if (n < 3) return pts;
    sort(pts.begin(), pts.end());
    vector<P> h(2 * n);
    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}
```